

Laboratory Notes

Week 2 Lab: Introduction to Linux and ROS

1 Aim

This laboratory aims to teach the student about the basics of Linux terminals and ROS. It will cover the following areas:

- Understanding of the Laboratory Setup, including Virtual Machines.
- Learn the basics of Linux terminals.
- Introduce basic ROS2 commands.
- Introduce the Turtlesim simulator.

2 Background

In this first lab we will be exploring the Linux Virtual Machine environment and use of the Linux terminal, including basic terminal commands that we will need throughout the subject. We will also look at basic commands in ROS2, including how to run Nodes in ROS2 and how to find the important information about running nodes and the topics they are using. Throughout this introduction we will use the “turtlesim” simulation tool, which is a lightweight simulator that is commonly used to demonstrate basic ROS concepts.

3 Procedure

3.1 Starting the Virtual Machine

As mentioned during Lecture 1, the majority of the work done during the Laboratory classes will be conducted on a Virtual Machine (VM). At the time of writing these notes, two different Virtual Machine options were still being explored, one using VirtualBox and the other VMWare. Ask the lab demonstrator for help in starting up the correct Virtual Machine.

3.2 Introduction to Linux

In this section we will explore the basics of the Linux operating system.

3.2.1 Folder Structure

Enter the following commands (in order) into a new terminal window to see how each command functions. You are welcome to explore further with these commands, the listing below is designed for basic understanding of moving between folders and analysing their structure. For further information on the operations performed by these commands, refer to the notes from the Week 1 Lecture.

```
1 ls
2 ls -l
3 cd ros2_ws
4 cd src
5 ls -l
6 cd ecte477
7 cd
8 cd ros2_ws/src/ecte477
9 pwd
```



Figure 1: Tux is a penguin and is the official mascot of the Linux kernel.

3.2.2 New Directories and Files

Enter the commands in the listing below. These are standard commands for making new directories and files, along with changing some file permissions. You should change the name of the new directory that you set-up to your own name (i.e. instead of “jeffs_scripts” use your own name).

```
1 cd ros2_ws/src/
2 mkdir jeffs_scripts
3 ls -l
4 cd jeffs_scripts
5 ls -l
6 touch hello_world
7 ls -l
8 chmod +x hello_world
```

3.2.3 Editing a simple file

After completing the previous tasks, the next step will be to create a simple file using a text editor. There are several text editors in linux, which you can invoke via the command line – it is recommended that you try to use the ‘vi’ editor. Please refer to the Week 1 lecture for information on how to use vi. Open the editor using the following commands (remember to change “jeffs_scripts” to whatever you called your own directory):

```
1 cd
2 cd ros2_ws/src/jeffs_scripts
3 vi hello_world
```

You can now insert the following text using the text editor – don’t forget to save and exit once you are finished editing:

```
1 while true
2 do
3     echo "Hello World $counter"
4     ((counter++))
5 done
```

Enter the following terminal commands to run your new script and then remove the script and directory after it has successfully run (again, remember to change “jeffs_scripts” to whatever you called your own directory):

```
1 ./hello_world
2 ctrl + c
3 cd ..
4 ls -l
5 rm -r jeffs_scripts
6 ls -l
```

3.3 Introduction to ROS

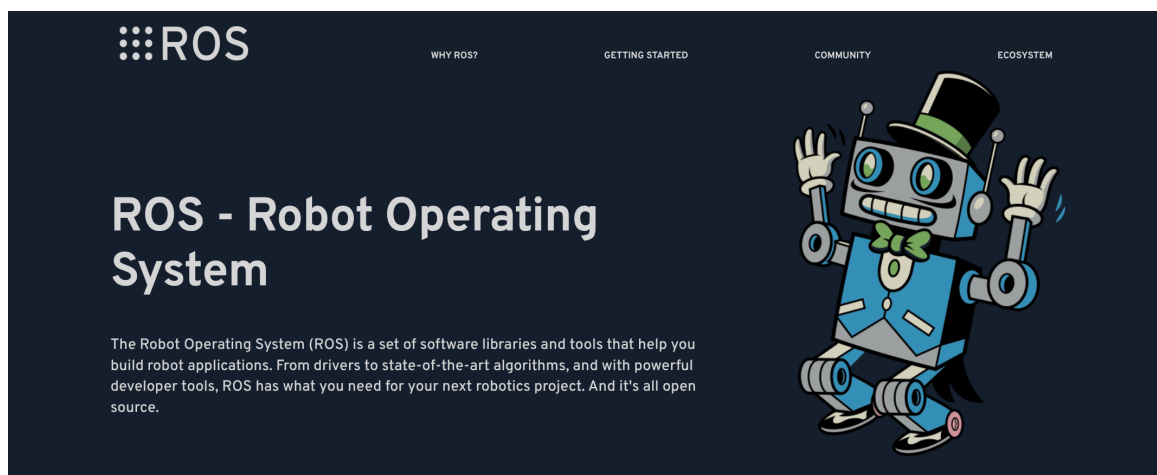


Figure 2: <http://www.ros.org>

In this section, we will use the lightweight ROS simulation tool called turtlesim to analyse and inspect some of the basic ROS concepts.

3.3.1 Teleoperation

Use the commands listed below to load the turtlesim tool and start the teleoperation ¹ node. In 2 separate terminals, launch these commands:

```
1 ros2 run turtlesim turtlesim_node
2 ros2 run turtlesim turtle_teleop_key
```

¹Teleoperation (or remote operation) indicates operation of a system or machine at a distance. It is similar in meaning to the phrase “remote control” but is usually encountered in research, academic and technical environments. It is most commonly associated with robotics and mobile robots but can be applied to a whole range of circumstances in which a device or machine is operated by a person from a distance. Source: Wikipedia

The separate terminals allow multiple commands to run simultaneously (you can also use tabs in your terminal app). By using the directional keys, you should now be able to move the turtle around inside turtlesim. You will need to make sure that the “turtle_teleop_key” terminal window has the input focus (i.e. is the active window), in order to input commands. Reset the simulation with the following command. A new terminal (or tab) will be needed:

```
1 ros2 service call /reset std_srvs/srv/Empty
```

Now close all running applications in your terminals by pressing ‘ctrl + c’ (in each terminal). Then exit your terminals by typing ‘exit’ in each terminal. All terminal windows should now be closed.

3.3.2 Inspecting ROS Packages

In this section we will explore how to interact with ROS packages. In a new terminal try the following commands:

```
1 ros2 pkg list
```

The ‘list’ command will list all the installed ROS packages on the system. To find the directory of a single package, use:

```
1 ros2 pkg prefix turtlesim
```

Note that ‘turtlesim’ was just chosen here as an example package. Once you have found the directory of a package you can use ‘cd’ to change to that directory. If you want to find the directory of a package without knowing the full name of the package you can also use tab completion ². For example, the following command will show you all package options that start with ‘turtle’ – you can then choose one from the list to find the directory of the specific package.

```
1 ros2 pkg prefix turtle [TAB] + [TAB]
```

3.3.3 ROS Nodes

Once started, a running instance of a ROS program is called a Node. We will have a closer look at ROS nodes using the Turtlesim Simulator. Relaunch turtlesim and turtle_teleop_key using the commands from Section 3.3.1, then enter the following command in a new terminal:

```
1 ros2 node list
```

This command should show that 2 nodes are running:

- /turtlesim: This node is responsible for instantiating the turtlesim window and simulating the motion of the turtle.
- /teleop_turtle: This node awaits an arrow key to be pressed, converts these key presses into a movement and then sends that command to the turtlesim node.

²An incredibly useful tool, tab-completion, is available in ROS. By pressing TAB ROS will auto-fill the command line. Tab-completion will only work for recognised system commands and ROS commands. If there are too many options for ROS to auto-fill the command line then pressing TAB a second time (in quick succession) will show all the available options.

We can inspect nodes with the following command:

```
1 ros2 node info /teleop_turtle
```

This command will output a list of topics that are related to that node, the services available and a summary of connections it has made to other nodes.

3.3.4 ROS Topics

Type the following command to show a list of active topics:

```
1 ros2 topic list
```

We will have a closer look at the topic `/turtle1/cmd_vel`. First, you can determine what message type is being used with this topic. This can be achieved with:

```
1 ros2 topic info /turtle1/cmd_vel
```

Once entered correctly, this command will show the message type being used as well as the publishers and subscribers to this topic.

3.3.5 RQT Graph

In order to visually see how ROS nodes & topics are interacting with each other, we can use a tool called `rqt_graph`. With `turtlesim` and teleoperation running, determine if a new terminal is required and enter:

```
1 rqt_graph
```

Once `rqt_graph` starts, the following settings are recommended:

- Choose Graph Type = Nodes/Topics (all)
- Group = 0
- Check Boxes = All should be ticked.

With these settings applied, the graph should look something like Figure 3 (although yours should also show the status and feedback of a `rotate_absolute` action). Note that you may need to use the “refresh” button in order to see the image. Here, we can see that the `/teleop_turtle` node is publishing the key strokes onto a topic called



Figure 3: Output from `rqt_graph`, showing how ROS Nodes and Topics interact with each other.

`/turtle1/cmd_vel`. The `/turtlesim` node then subscribes to the same topic to receive the key stroke command. Please now spend some time becoming familiar with this tool by changing the various options to see their effect on the graph. `rqt_graph` is incredibly useful to visualise your running nodes and their communication.

Points to note:

- The `/turtlesim` node does not know, or even care, who publishes the `/cmd_vel` messages. Any program that publishes onto that topic can control the turtle.
- The `/teleop_turtle` doesn't know or care who subscribes to the `/cmd_vel` messages that it publishes. Any program that subscribes to that topic is free to respond to those commands.

Now close `rqt_graph`

3.3.6 ROS Messages:

As noted on the ROS Wiki, Nodes communicate with each other by publishing messages to topics. A message is a simple data structure, comprising typed fields. Standard primitive types (integer, floating point, boolean, etc.) are supported, as are arrays of primitive types. Here, we will have quick look at what message type is being used with the `/turtle1/cmd_vel` topic:

```
1 ros2 topic type /turtle1/cmd_vel
```

To see the fields within the message type, use the following command. You will need to replace 'message_type' with the message type identified in the previous command:

```
1 ros2 interface show message_type
```

It is possible to publish data to a topic directly from the command line. Assuming `turtlesim` is still running, try entering the following command (note that it should be entered on one single line):

```
1 ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}"
```

Here we have set the "{linear: {x:" value to 2.0, which tells the turtle to move forward with a command velocity of 2.0. Repeat this command to publish various linear and angular values and see how the `turtlesim` functions. Try to get the turtle to rotate.

3.3.7 Inspecting Messages

Ensure that the `turtlesim` simulator and `teleop_key` are running. By using the command 'ros2 topic echo', we can see what data is being published on a topic:

```
1 ros2 topic echo /turtle1/cmd_vel
```

As nothing is currently being published, you should not be seeing any values. Now make sure the active window is the teleoperation window, start moving the turtle around. In the terminal window running 'ros2 topic echo', you should now see the data that is being published on the `/turtle1/cmd_vel` topic. Using the command line, publish a command to the `turtlesim` to make the turtle move. In the terminal running `ros2 topic echo`, you should be able to see the values that were sent over the topic `/turtle1/cmd_vel`. Try publishing a variety of values to see how the system is working.

We will now publish constantly, at a rate of 1 hz. Using the same publishing command, include the option “-rate” (as shown below). Choose appropriate linear and angular values to make the turtle drive in a circle.

```
1 ros2 topic pub --rate 1 /turtle1/cmd_vel .....
```

In the ros2 topic echo terminal, you can see the values that are constantly being published. To inspect the rate at which the values are being published:

```
1 ros2 topic hz /turtle1/cmd_vel
```

3.4 Additional Commands

Try these additional Linux commands:

Check Ubuntu version

```
1 lsb_release -a
```

Check version of ROS installed:

```
1 cd /opt/ros
2 ls
```

4 Summary

In today’s laboratory we have explored some of the basics of Linux and ROS. Many of the commands used today will be incredibly useful in the labs to follow. Next week we will look at the turtlebot3 models in ROS, along with the Gazebo simulation application, and developing simple ROS nodes in Python.